HiPPO Results Report

What This Short Report Covers

This version keeps the practical part of the work:

- what datasets were run
- what results were obtained
- what the benchmark comparison showed

The long setup story was removed because it is not the main focus right now.

Datasets That Were Run

During this work, three datasets were run:

1. `mnist`
2. `adding`
3. `copying`

If only two datasets are needed for the report, the best two to mention are:

- `MNIST`
- `Copying`

This is because:

- `MNIST` is the main real dataset that people know
- `Copying` is the dataset used for the 4-model comparison

What MNIST Is

MNIST is a dataset of small black-and-white images of handwritten digits. The digits are 0, 1, 2, and so on up to 9.

The model looks at each image and tries to predict which digit it is.

In this project, `mnist` is the default dataset in the config, so when the project is run normally, it tries to use MNIST first.

MNIST Result

After fixing the environment and manually providing the MNIST files, the project was run successfully on MNIST.

The result was:

- training accuracy: `0.848`
- validation accuracy: `0.956`
- test accuracy: `0.961`

What The MNIST Result Means

This means the model learned the task successfully.

The most important number is the test accuracy:

- `0.961` means about `96.1%` of the test images were classified correctly

In simple words:

- the model was able to recognize handwritten digits well
- the training pipeline worked
- the GPU setup also worked

So MNIST is the clearest example showing that the project can run and produce a meaningful result.

What Adding Is

`Adding` is not downloaded from the internet. It is an artificial dataset generated by the code itself.

The task is roughly this:

- the model sees a sequence of numbers

- it must learn which values matter
- then it must produce the correct sum

This dataset is used to test whether the model can handle long sequences and keep useful information.

Adding Result

The `adding` dataset was run successfully.

The visible final result from the run was:

- loss: about `0.171`

What The Adding Result Means

For this dataset, the main value is loss, not accuracy.

In simple words:

- lower loss is better
- a loss of about `0.171` shows the model was learning the task

So the `adding` run shows that the project also worked on a synthetic sequence dataset, not only on image data.

What Copying Is

`Copying` is also an artificial dataset generated inside the project.

This task checks sequence memory.

In simple words, the model must:

- read a sequence
- remember important information from earlier in the sequence
- reproduce that information later

This is useful because it tests how well different model types handle memory over time.

Sequence Model Comparison Setup

After getting the project to run, a comparison setup was added so different sequence-model families could be tested in the same training pipeline.

The following model families were used:

- `gru` for **recurrence**
- `conv1d` for **convolution**
- `attention` for **attention**
- `legs` for **state-space / HiPPO**

Important note:

This repository does **not** contain a true S4 implementation. The closest built-in state-space model available here is `legs`, which is HiPPO-based. So in this project, the state-space comparison was done with `legs`, not with a full S4 model.

Benchmark Script

To avoid running every benchmark manually, a script was added:

- `run_copying_benchmarks.py`

This script:

1. runs all four models on the `copying` dataset
2. keeps the training settings consistent
3. saves the full logs
4. extracts runtime and final metrics into result files

Generated output files:

- `benchmark_outputs/copying_benchmark_results.json`
- `benchmark_outputs/copying_benchmark_results.md`

Benchmark Dataset Choice

The `copying` dataset was used for the architecture comparison because:

- it is built into the repository
- it does not need internet download
- it is a sequence task
- it is short enough that the attention model can run on the available GPU

The training command pattern used by the benchmark script was based on:

- `dataset=copying`
- `dataset.num_workers=0`
- `train.epochs=2`

The `dataset.num_workers=0` setting was kept because this machine is Windows-based and the project otherwise runs into multiprocessing problems.

Copying Benchmark Results

The following results were collected from the automated benchmark run:

Model	Family	Runtime (s)	Train Acc	Val Acc	Test Acc
`gru`	Recurrence	485.03	0.109	0.009	0.008
`conv1d`	Convolution	28.67	0.104	0.101	0.099
`attention`	Attention	44.63	0.126	0.123	0.125
`legs`	State-space / HiPPO	639.89	0.094	0.132	0.130

What These Benchmark Results Mean

These numbers compare two things:

- how long the model took to run
- how well the model performed on the `copying` task

Runtime

Runtime means training time.

From the results:

- `conv1d` was the fastest: `28.67` seconds
- `attention` was also fairly fast: `44.63` seconds
- `gru` was much slower: `485.03` seconds
- `legs` was the slowest: `639.89` seconds

In simple words:

- convolution was the fastest model here
- attention was also efficient
- recurrence and HiPPO took much longer in this experiment

Accuracy

Accuracy tells us how often the model gave the correct answer.

The most useful value here is test accuracy:

- `gru`: `0.008`
- `conv1d`: `0.099`
- `attention`: `0.125`
- `legs`: `0.130`

In simple words:

- `legs` gave the best test result in this benchmark
- `attention` was close behind
- `conv1d` was lower
- `gru` performed poorly in this short run

Important Note About These Comparison Results

These benchmark results should be explained carefully.

They do **not** mean:

- convolution is always best
- recurrence is always bad
- attention is always best
- HiPPO is always best

Why not:

- all models were run only for `2` epochs
- this was a short comparison
- no deep tuning was done for each model

So the fair conclusion is:

> In this benchmark, convolution was the fastest model, while the HiPPO-based `legs` model achieved the best test accuracy on the `copying` task.

Final Overall Conclusion

The most important practical results from this work are:

1. The project was run successfully on `MNIST`
2. The project was also run on synthetic sequence datasets such as `adding` and `copying`
3. A 4-model comparison was completed on the `copying` dataset

The clearest result examples are:

- `MNIST` test accuracy: `0.961`
- `Adding` loss: about `0.171`
- `Copying` benchmark: `conv1d` fastest, `legs` best test accuracy

So if the report needs a simple summary, it can say:

> The project was successfully run on MNIST and on sequence datasets generated by the code. MNIST gave a strong result of about 96.1% test accuracy. On the Copying benchmark, four model families were compared, and the fastest model was convolution, while the best test accuracy was achieved by the HiPPO-based `legs` model.